ScriptLens

Final Technical Architecture

High-Level Design | Low-Level Design | Full Verdict

Product	AI-powered screenplay intelligence layer
Format	Chrome Extension — floating orb + side-by-side popup panel
Core features	Scene dependency graph + Plot contradiction engine
Architecture	Extension service worker + FastAPI backend + Postgres
Target	8GB+ RAM machines — footprint under 26MB
Version	2.3 — Adds scroll-to-scene navigation + 40-word scratchpad limit
Date	May 2026
New in v2.3	Scroll-to-scene: clicking a result scrolls the editor to that scene. Scratchpad capped at 40 words / ~200 characters. sceneAnchorMap built on popup open.

1. Executive Summary

ScriptLens v2.3 is a Chrome extension that floats above any screenplay writing environment as a small interactive orb. It analyses screenplay text for structural dependencies between scenes and logical contradictions within the narrative. The extension is platform-agnostic, unbreakable by host editor updates, and runs core analysis in under 800ms.

Version 2.3 adds two changes to v2.2. First, the popup panel gains scroll-to-scene navigation: when a writer clicks any dependency or contradiction result, the host editor scrolls to the scene in question. This closes the most important gap in the v2.2 experience — the writer could see a problem but had to manually find it in their script. Now they tap and land. Second, the scratchpad word limit is capped at 40 words (approximately 200 characters). This keeps the scratchpad as a quick-thought tool rather than a second editor.

The scroll-to-scene feature adds one new component (SceneAnchorMap builder), one new message type (SCROLL_TO_SCENE), and zero backend changes. No new tables. No new endpoints. No change to Zone B or Zone C.

Dimension	v2.2	v2.3 change	Impact
Scroll to scene	Not available — writer finds scene manually	Click result → editor scrolls to scene	New content script function
sceneAnchorMap	Does not exist	Built once on popup open from DOM	New data structure, content script only
SCROLL_TO_SCENE message	Does not exist	New message: popup → SW → content script	New message type, extension only
Scratchpad word limit	Unlimited text input	40 words max (~200 chars), live counter	maxLength on textarea + word counter UI
Scratchpad label	'Your notes — cleared when you close'	Same label + word counter: '0/40 words'	UI addition only
Backend changes	—	None	Zones B and C unchanged
Database changes	—	None	Schema unchanged
RAM footprint	~24MB	~24.5MB (anchor map ~0.5MB on long scripts)	Negligible
New API endpoints	—	None	No new endpoints

2. System Overview

2.1 The Three Zones

Unchanged from v2.2 in structure. Zone A (Chrome extension) gains two additions. Zones B and C are identical to v2.2.

ZONE A — Chrome Extension (capture + render, no analysis) — 2 additions in v2.3

- Content script: text extraction, change detection, orb rendering. v2.3 adds: SceneAnchorMap builder (runs once when popup opens) and scrollToScene() handler for SCROLL_TO_SCENE messages.
- Service worker: state machine, Fountain parsing, API client. v2.3 adds: SCROLL_TO_SCENE message routing — receives from popup, forwards to content script on the active tab.
- Orb: 52px canvas. HSL colour-fade in WATCHING state. Unchanged.
- Popup panel: side-by-side layout. Left: scratchpad (40-word limit, ephemeral). Right: Dependencies and Contradictions tabs. v2.3 adds: click handlers on every result item that dispatch SCROLL_TO_SCENE. Word counter on scratchpad.

ZONE B — FastAPI Backend (all intelligence — unchanged from v2.2)

- Scene ingestion + spaCy NLP. Dependency graph (NetworkX). 3-tier contradiction engine. Fact store. Redis cache. All unchanged.

ZONE C — Third-Party Services (unchanged from v2.2)

- Supabase, Upstash Redis, Anthropic API (Haiku), Fly.io, Stripe. All unchanged.

3. High-Level Design (HLD)

3.1 Data Flow — Live Writing Session

Steps 1–16 unchanged from v2.2. Steps 17–21 reflect v2.3 popup and scroll behaviour.

1. Writer edits script. MutationObserver fires.
2. 1.5s debounce. extractText + hashText.
3. OrbFadeEngine.increment(). Posts to service worker if hash changed.
4. Service worker: Fountain.js parse. POST to FastAPI /analyse.
5. FastAPI: Redis check → full analysis pipeline if cache miss.
6. Result returned. chrome.storage.local updated. Edit counter reset.
7. Orb: RESULTS_READY state. Amber pulse.
8. Writer clicks orb. Popup opens immediately.
9. Popup onMount: calls chrome.runtime.sendMessage({ type:'GET_ANCHOR_MAP' }).
10. Content script receives GET_ANCHOR_MAP. Runs buildSceneAnchorMap(). Returns map to popup via response callback.
11. Popup stores anchor map in local React state. Reads analysis result from chrome.storage.local. Renders side-by-side layout.
12. Left column: scratchpad textarea, maxLength 200 chars, live word counter '0/40 words'.
13. Right column: Dependencies tab or Contradictions tab. Each result item has a 'Go to scene' button.
14. Writer reads a contradiction. Clicks 'Go to scene' on Scene 12.
15. Popup dispatches chrome.runtime.sendMessage({ type:'SCROLL_TO_SCENE', sceneId:'scene_012', yOffset: anchorMap['scene_012'] }).
16. Service worker receives SCROLL_TO_SCENE. Forwards to content script on active tab via chrome.tabs.sendMessage.
17. Content script receives SCROLL_TO_SCENE. Calls scrollToScene(yOffset). Editor scrolls smoothly to Scene 12.
18. Writer closes popup. Scratchpad text destroyed. Orb returns to WATCHING state.

3.2 Orb State Machine — unchanged from v2.2

State	Colour	Animation	Entry	Exit
WATCHING	Teal fade (pale→saturated)	HSL fade only	Extension load or popup closed	Edit threshold OR 4min idle
ANALYSING	Purple #7F77DD	Slow rotation ring	Hash changed + POST sent	Backend responds OR 8s timeout
RESULTS_READY	Amber #BA7517	Gentle 2s pulse	Backend response received	Writer opens popup
DORMANT	Gray #888780	Fade to 40% opacity	No edit for 240s	Any keystroke

State	Colour	Animation	Entry	Exit
COOLING	Red #E24B4A	Static dim	3 opens in 90s	60s timer expires

3.2a Orb Colour-Fade — unchanged from v2.2

Full specification in v2.2 document. Summary: HSL(168, 20+I*60, 72-I*32). editCount 0→12. fadeThreshold configurable. Paste = 1 edit. Undo does not decrement.

3.3 Scroll-to-Scene Navigation

NEW IN v2.3

When a writer clicks any result in the analysis panel — a scene dependency item or a contradiction flag — the host editor scrolls smoothly to that scene. This is the feature that turns ScriptLens from a reading tool into a navigation tool. The writer does not need to remember a scene number and manually find it. They tap once and land.

> Architecture principle: the popup knows scene IDs and Y-offsets (from the anchor map). The content script owns the scroll action. These never swap responsibilities. The popup sends a message; the content script executes the scroll. Zero DOM knowledge in the popup.

How the anchor map is built

On every popup open, a GET_ANCHOR_MAP message is sent to the content script. The content script scans the host editor's DOM to find each scene heading element and records its getBoundingClientRect().top + window.scrollY value — the absolute Y position of that scene on the page. This scan takes under 20ms on a 120-scene script.

The anchor map is a plain object: { scene_id: yOffset }. For example: { 'scene_001': 0, 'scene_002': 842, 'scene_003': 1620, ... }. It is stored in popup React state only. It is rebuilt on every popup open — never cached — so it stays accurate if the writer has scrolled or the page has reflowed.

How scene headings are found in the DOM

The content script uses a CSS selector strategy, not WriterDuet-specific selectors. It searches for text nodes that match Fountain scene heading patterns: text starting with INT. or EXT. followed by a space. This works on any editor that renders screenplay text in the DOM, including WriterDuet, Arc Studio, and Google Docs with Fountain-formatted content.

Editor	How headings are found	Fallback if fails
WriterDuet	Text nodes matching /^(INT|EXT)\./i within the editor contenteditable div	Return empty anchor map. Scroll buttons show 'Scene not locatable' tooltip.
Arc Studio	Same text-node strategy — Arc renders headings as plain text	Same fallback
Google Docs	Text nodes in .kix-paragraphrenderer spans	Same fallback
Fountain .txt file (upload mode)	Not applicable — no DOM editor present	Scroll button hidden entirely for upload mode

Scroll behaviour

- Smooth scroll using window.scrollTo({ top: yOffset - 80, behavior: 'smooth' }). The -80px offset keeps the scene heading visible with breathing room above it.
- If yOffset is undefined (scene not in anchor map): show a tooltip on the button: 'Scene not locatable in editor — check your cursor position.'
- If the popup is open during the scroll, it stays open. The writer can click another scene immediately.
- Scroll is triggered from the popup, executed in the page. The popup iframe cannot scroll the host page directly — the message-passing architecture is required.

3.4 Popup Panel — Side-by-Side Layout v2.3

CHANGED v2.3

Layout identical to v2.2. Two changes: scratchpad has 40-word limit with live counter. Each result item in the analysis column has a 'Go to scene' button.

Element	v2.2 spec	v2.3 change
Popup width	760px	Unchanged
Left column	340px scratchpad, no limit	340px scratchpad, maxLength 200 chars
Scratchpad label	'Your notes — cleared when you close'	Label unchanged. Below it: live word counter '0 / 40 words' in rgba(255,255,255,0.35)
Scratchpad textarea	Uncontrolled, no limit	Controlled: value state tracked for word count display. maxLength=200. onKeyDown blocks input at 40 words.
Word counter	Does not exist	Inline counter below textarea. Updates on every keystroke. Turns amber at 35+ words. Turns red at 40 words.

Element	v2.2 spec	v2.3 change
DependencyTab result item	Scene heading + dependency count badge	+ 'Go to scene' button (small, right-aligned). Dispatches SCROLL_TO_SCENE on click.
ContradictionCard	Excerpt + explanation + confidence + tier badges	+ 'Go to scene A' and 'Go to scene B' buttons for both involved scenes.
ImpactPopover item	Scene heading text	+ 'Go to scene' button on each impacted scene listed

40-word scratchpad logic

The word count is computed as: text.trim().split(/\s+/).filter(w => w.length > 0).length. At 40 words, new words are blocked via onKeyDown — only deletion keys (Backspace, Delete) and navigation keys are permitted. The counter display is: '37 / 40 words' in amber when above 35. '40 / 40 words' in red with slight shake animation when limit is hit.

3.5 Feature 1 — Scene Dependency — unchanged

Dependency type	Weight	Detection	Example
Character reappearance	1.0	Named entity match (spaCy)	Marcus introduced Sc.2 — appears Sc.7,14,22
Object reference	0.7	Noun phrase extraction	Briefcase introduced Sc.3 — referenced Sc.11
Established fact	0.6	Fact store lookup	Hospital established Sc.1 — assumed Sc.9
Causal dialogue	0.5	Semantic similarity	'After what you did yesterday' — requires prior scene
Location continuity	0.4	Entity + scene heading	INT. SAME ROOM — depends on preceding scene

3.6 Feature 2 — Plot Contradiction — unchanged

Tier	Technology	Confidence	Cost	Accuracy
1 — Deterministic rules	Python rule engine	0.95	Zero	98% precision, 62% recall
2 — NLP semantic	spaCy en_core_web_sm	0.75	Zero	91% precision, 88% recall
3 — LLM reasoning	Claude Haiku 3.5	0.60	~$0.03/call	97% precision, 94% recall

4. Low-Level Design (LLD)

4.1 Chrome Extension — Module Breakdown

4.1.1 Content Script — v2.3 additions

CHANGED v2.3

Two new functions added. All existing functions unchanged. File grows from ~180 lines to ~240 lines.

Function	Signature	Description	Side effects
buildSceneAnchorMap	() => Record<string, number>	Scans document for text nodes matching /^(INT|EXT)\./i. For each match, walks up DOM to find the block element. Records getBoundingClientRect().top + window.scrollY as yOffset. Returns { scene_id: yOffset } object. scene_id derived from scene number in order of appearance.	DOM read — no writes. Runs in < 20ms.
scrollToScene	(yOffset: number) => void	Calls window.scrollTo({ top: yOffset - 80, behavior: 'smooth' }). If yOffset is undefined or NaN, does nothing and logs warning.	Scrolls host page. No DOM mutation.
Message listener addition	chrome.runtime.onMessage	Handles two new message types: GET_ANCHOR_MAP (calls buildSceneAnchorMap, returns result via sendResponse) and SCROLL_TO_SCENE (calls scrollToScene with yOffset from message).	Registered once on content script load.

4.1.2 Service Worker — v2.3 additions

One new message type added to the existing onMessage handler. All existing modules unchanged.

Message type	Direction	Handler	Side effect
GET_ANCHOR_MAP	popup → SW →	SW receives from popup. Calls chrome.tabs.sendMessage(activeTabId,	Tab ID lookup via chrome.tabs.query

Message type	Direction	Handler	Side effect
	content script	{ type:'GET_ANCHOR_MAP' }) and pipes response back to popup.	
SCROLL_TO_SCENE	popup → SW → content script	SW receives from popup: { type:'SCROLL_TO_SCENE', sceneId, yOffset }. Forwards to active tab content script via chrome.tabs.sendMessage.	No storage write. Fire and forget.

4.1.3 SceneAnchorMap Builder — detail

NEW IN v2.3

The anchor map builder must handle the fact that different editors render scene headings differently. The strategy uses text content matching, not CSS class matching, so it is resilient to editor UI updates.

Step	Code pattern	Notes
1. Get all text nodes	TreeWalker with SHOW_TEXT filter	Walks entire document body. Skips script, style, noscript elements.
2. Filter for scene headings	node.textContent.trim().match(/^(INT\.|EXT\.)/i)	Case-insensitive. Matches INT. INT/EXT. EXT. and variants. Does not match INTERNAL or EXTERIOR.
3. Find block element	Walk up parentNode until display is block	Finds the containing paragraph/div that holds the heading text.
4. Compute Y position	blockEl.getBoundingClientRect().top + window.scrollY	Absolute page position regardless of current scroll.
5. Assign scene_id	'scene_' + String(index+1).padStart(3,'0')	scene_001, scene_002, etc. Must match scene_ids produced by Fountain.js parser in service worker.
6. Return map	{ 'scene_001': 0, 'scene_002': 842, ... }	Plain object. No class instances. Serialisable for message passing.

Critical: the scene_id numbering must match exactly between the SceneAnchorMap (built from DOM) and the SceneBlock array (built by Fountain.js parser in SW). Both use sequential numbering from 1. If a scene has no heading visible in the DOM (e.g. page not scrolled into view), getBoundingClientRect() may return 0 — always add window.scrollY to compensate.

4.1.4 Popup Panel — v2.3 components

CHANGED v2.3

Component	Change in v2.3	New props/state
App	On mount: sends GET_ANCHOR_MAP, stores result in anchorMap state. Passes anchorMap to AnalysisColumn.	anchorMap: Record<string,number> state
ScratchpadColumn	Now controlled component for word count. maxLength 200. Word counter below textarea.	wordCount: number state. onTextChange handler.
ScratchpadTextarea	Controlled: value + onChange. maxLength=200 attr. onKeyDown blocks at 40 words.	value, onChange, wordCount props
WordCounter	New component. Shows 'N / 40 words'. Amber > 35. Red at 40. Shake animation at limit.	wordCount: number prop
DependencyTab	Each scene item gains 'Go to scene' button. Calls onScrollRequest(sceneId).	onScrollRequest: (id:string)=>void prop
ImpactPopover	Each impacted scene gains 'Go to scene' button.	onScrollRequest prop passed through
ContradictionCard	'Go to scene A' button (scene_id_a) and 'Go to scene B' button (scene_id_b).	onScrollRequest prop. Two buttons per card.
ScrollButton	New shared component. Small button: arrow icon + 'Scene N'. Disabled state if sceneId not in anchorMap.	sceneId, sceneNumber, anchorMap, onScrollRequest props

4.1.5 TypeScript Types — v2.3

One new type added. All existing types unchanged from v2.2.

Type	Fields	Status
SceneAnchorMap (NEW)	Record<string, number> — { scene_id: yOffset }	New in v2.3. Built by content script. Stored in popup state only.
ScrollToSceneMessage (NEW)	{ type: 'SCROLL_TO_SCENE', sceneId: string, yOffset: number }	New message type. Popup → SW → content script.

Type	Fields	Status
GetAnchorMapMessage (NEW)	{ type: 'GET_ANCHOR_MAP' }	New message type. Popup → SW → content script.
SceneBlock	Unchanged	id, heading, type, characters[], action[], dialogue[], objects[], hash
DependencyEdge	Unchanged	fromSceneId, toSceneId, weight, edgeType, explanation
Contradiction	Unchanged	id, sceneId, excerpt, conflictsWith, conflictSceneId, confidence, tier, type, explanation
AnalysisResult	Unchanged	scriptId, timestamp, scenes, dependencies, contradictions, healthScore
FadeState	Unchanged	editCount, editVelocity, fadeIntensity, lastEditAt

4.2 Backend — all unchanged from v2.2

The scroll-to-scene feature is entirely client-side. The backend has zero changes in v2.3. All endpoints, all models, all database tables, all pipeline steps are identical to v2.2. For full backend LLD see v2.2 document sections 4.2.1 through 4.2.7.

Backend component	v2.3 status
POST /analyse	Unchanged — returns AnalysisResult with dependencies and contradictions
POST /scene/delete-impact	Unchanged
GET /script/history	Unchanged
spaCy pipeline	Unchanged — NER, dep, sents
NetworkX graph engine	Unchanged — DiGraph, nx.descendants, nx.ancestors
3-tier contradiction engine	Unchanged — deterministic rules, semantic, Haiku
Postgres schema	Unchanged — no new tables
Redis cache	Unchanged — scene hash TTL 24h
Supabase Auth	Unchanged
Stripe billing	Unchanged

5. Performance and Reliability

5.1 Latency Budget — v2.3 additions

Step	p50	p95	Failure mode	Mitigation
buildSceneAnchorMap (NEW)	< 15ms	< 40ms	Editor DOM not yet loaded	Retry after 200ms. Return empty map if 3 retries fail.
GET_ANCHOR_MAP message round-trip (NEW)	< 25ms	< 60ms	Content script not responding	Return empty map. ScrollButton shows disabled state.
SCROLL_TO_SCENE round-trip (NEW)	< 10ms	< 30ms	Tab closed between click and scroll	Catch error silently. Log to console.
scrollToScene() execution (NEW)	< 5ms	< 15ms	yOffset undefined	Guard clause: if(!yOffset) return. Log warning.
Word counter update (NEW)	< 1ms	< 2ms	None	Trivial string split operation.

All existing latency budgets from v2.2 (hash check, Fountain parse, FastAPI call, Redis, spaCy, Haiku) are unchanged. Total analysis pipeline SLA: under 800ms p95 for cache miss without Tier 3.

5.2 RAM Profile — v2.3

Component	RAM	Notes
SceneAnchorMap (120 scenes)	~0.5 MB	Plain JS object. 120 string keys + 120 numbers. Rebuilt on each popup open, not persisted.
Word counter state	< 0.1 MB	Single integer. Negligible.
All other components	Unchanged from v2.2	Total ~24MB active (popup open)
TOTAL (popup open)	~24.5 MB	Down from v2.1 (26MB), up 0.5MB from v2.2 (24MB)

5.3 Reliability — v2.3 additions

Failure	Detection	Degradation	Recovery
buildSceneAnchorMap returns empty (editor not loaded)	anchorMap === {} in popup state	ScrollButton renders in disabled state with tooltip: 'Scene not locatable — editor may still be loading.'	Popup re-requests anchor map if writer keeps it open > 2s after open

Failure	Detection	Degradation	Recovery
Scroll offset wrong (editor reflowed after map built)	No detection — silent wrong scroll	Writer lands near the scene but not exactly on it. Acceptable degradation — they can see the heading.	Map is rebuilt on every popup open, so next open corrects it
Writer pastes 300 chars in scratchpad (bypasses maxLength)	Word count > 40 detected on paste	onPaste handler truncates to 200 chars and updates wordCount	Paste handler reads clipboard text, slices to first 200 chars, sets value
SCROLL_TO_SCENE arrives after tab navigated away	chrome.tabs.sendMessage throws	Caught silently. No user-visible error.	Not recoverable — expected edge case

6. Security Design — v2.3

v2.3 adds one new security consideration: the anchor map builder reads the host page DOM. This is consistent with what the content script already does (text extraction) but is worth noting explicitly.

Concern	Risk	Mitigation	Disclosure?
Anchor map reads host DOM (NEW)	Content script reads getBoundingClientRect on DOM elements	Read-only. No data sent anywhere. Anchor map stored in popup iframe memory only. Destroyed on popup close.	No — consistent with existing text extraction
SCROLL_TO_SCENE modifies host page scroll	Content script calls window.scrollTo on host page	This is the intended behaviour. It is disclosed in the Chrome Web Store listing under 'Permissions used'.	Yes — Chrome Web Store listing
Script text to server	Sensitive creative content	Disclosed in onboarding.	Yes
Haiku excerpts	Up to 500 chars sent to Anthropic	Opt out in settings.	Yes — privacy policy
Scratchpad text	Writer's private notes	Never leaves browser. Auto-destroyed.	Label in UI
JWT tokens	In chrome.storage.local	Isolated per extension. 1hr expiry.	No — standard

7. Build Sequence — v2.3

Scroll-to-scene and the scratchpad word limit are added to Phase 4 (dependency panel). Both are small additions — combined they add 2–3 days to that phase. No other phases change.

Phase	Duration	What to build	v2.3 addition	Completion signal
0	Week 1	Foundation: Plasmo, Supabase, Fly.io, FastAPI, Stripe, domain	None	FastAPI /health returns 200
1	Weeks 1–2	Fountain.js parser. 10 real scripts validated.	None	100 scenes parsed from 5 formatting styles
2	Week 2	Orb baseline: 5-state FSM, canvas, state machine	None	All 5 states transition correctly
2a	Week 2 add-on	Orb colour-fade: OrbFadeEngine, HSL fade, editCount	None	Orb fades across 12 edits over 90s
3	Weeks 3–4	Dependency backend: spaCy, fact store, NetworkX, /analyse	None	Delete sc.5 → correct impact list on 3 scripts
4	Week 4	Dependency panel + side-by-side popup layout + scratchpad	Word counter (40-word limit) + ScrollButton + buildSceneAnchorMap + SCROLL_TO_SCENE messages	Writer sees impact list. Click 'Go to scene' scrolls editor. Scratchpad blocks at 40 words.
4a	Weeks 5–7	Contradiction T1+T2: 5 rules, spaCy semantic, ContradictionTab	'Go to scene A' + 'Go to scene B' buttons on each ContradictionCard	FP rate < 10%. Clicking contradiction scrolls to correct scene.
5	Week 7	Contradiction T3: Haiku batch, 90s batching, cache	None	Haiku resolves 5 known ambiguous cases
6	Week 8	Auth + billing: Supabase Auth, Stripe, feature gating	None	Payment succeeds. Features gate correctly.
7	Week 9	Gothic health card: canvas, before/after, PNG export	None	1200x800 PNG exports cleanly
8	Weeks 10–11	Polish + launch: errors, onboarding, Chrome Web Store	None	Extension listed

8. Cost Projections — v2.3

Scroll-to-scene and the word counter add zero infrastructure cost. They are pure client-side features. Cost profile identical to v2.2.

Service	0–100 users	100–500 users	500–2000 users	v2.3 delta
Fly.io	$0	$5/mo	$20/mo	None
Supabase	$0	$0	$25/mo	None
Upstash Redis	$0	$0	$10/mo	None
Claude Haiku	$0–2/mo	$5–15/mo	$40–80/mo	None
Vercel	$0	$0	$0	None
Stripe	2.9%+$0.30	2.9%+$0.30	2.9%+$0.30	None
TOTAL	$0–2/mo	$5–20/mo	$95–130/mo	Unchanged from v2.2

9. Final Verdict — v2.3

SCROLL-TO-SCENE IS THE FEATURE THAT MAKES SCRIPTLENS USEFUL RATHER THAN INTERESTING.

9.1 Why scroll-to-scene changes the product

- **Without scroll-to-scene: the writer sees 'Scene 47 contradicts Scene 12 on character occupation' and then spends 90 seconds manually scrolling a 120-page script to find Scene 47. The analysis told them something but forced them to do the work of acting on it. This is the single biggest UX failure of v2.2.**
- **With scroll-to-scene: they click 'Go to Scene 47' and land there in under 200ms. The cognitive load of linking insight to location is eliminated. The writer stays in flow. ScriptLens stops being a separate tool and starts feeling like an extension of the editing environment.**
- **The architecture change is minimal — one new data structure, one new message type, two new content script functions. The impact on user experience is disproportionately large. This is the best ratio of implementation cost to UX value in the entire ScriptLens feature set.**

9.2 What v2.3 gets wrong or risks

- **The anchor map builder assumes scene headings are rendered as accessible text nodes. Some editors render headings in canvas elements (rare but possible) or behind virtual scrolling (Notion-style). In these cases the anchor map will be empty and scroll buttons will be disabled. Test on WriterDuet, Arc Studio, and Highland 2 specifically before launch.**
- **The anchor map is built once on popup open and never updated. If the writer edits the script while the popup is open — adding a scene above Scene 47 — the offsets become stale. The writer closes and reopens the popup to refresh. This is acceptable but should be documented in onboarding.**
- **The 40-word scratchpad limit will frustrate exactly one type of writer: the one who wants to write detailed notes about multiple scenes simultaneously. For that writer, the scratchpad is the wrong tool — they need a separate notes application. The limit enforces the design intent (quick thought, not a document) but will generate complaints. Prepare a response: 'The scratchpad is a quick-thought space. For longer notes, your script editor's notes panel is the right place.'**

9.3 Unchanged truths — still non-negotiable

- Fountain.js parser resilience. Every v2.3 feature — scroll, fade, scratchpad — is downstream of correct parsing. One week of parser testing on real-world messy scripts before building anything else.

- False positive discipline. A scroll-to-scene button on a wrong contradiction flag is more damaging than no button — it wastes the writer's time with one tap instead of making them search manually. The 6% false positive target is a hard gate.
- Community presence before launch. The scroll-to-scene feature is demonstrable in a 10-second screen recording. Make that recording. Post it in r/Screenwriting and Coverfly forums before you launch.

9.4 One-sentence verdict

v2.3 turns ScriptLens from a tool that tells writers what is wrong into a tool that takes them there — and that is the difference between a product writers keep installed and one they uninstall after the first session.

End of Document — ScriptLens Final Technical Architecture v2.3

The orb fades. The writer thinks. They click. They land. The script gets better.